Alexandria University

Faculty of Computers and Data Science

Course: Intelligent Programming

Task 1: Search Algorithms

Warehouse Delivery Robot

Team Members:

1. [Abdelrahman Hamdy] ID: [2403244068]
2. [Hana Elgaml] ID: [2403248169]
3. [Malak Hatem] ID: [2403243572]

1 Problem Formulation

The problem consists of navigating a robot through a warehouse grid to reach a target destination. We can formalize the problem as follows:

- **State Space:** Any valid cell coordinate (r, c) within the grid that is not a wall (#).
- **Initial State:** The starting cell coordinate S.
- **Goal State:** The goal cell coordinate G.
- **Actions:** The robot can move Up, Down, Left, or Right (one cell at a time).
- **Transition Model:** Moving in a valid direction updates the robot's coordinates to the adjacent cell.
- **Action Cost:** Moving to a normal floor (.) costs 1. Moving to a rough floor (~) costs 3. Entering the Goal (G) costs 1.
- **Goal Test:** Checks if the current state (r, c) matches the coordinates of G.

2 Algorithm Descriptions

2.1 Uniform Cost Search (UCS)

Uniform Cost Search is an uninformed search algorithm that guarantees finding the optimal path by expanding the node with the lowest path cost $g(n)$ from the start node. It uses a priority queue to always explore the cheapest available option.

```
# UCS Core Logic Snippet
while frontier:
    cost, _, current = heapq.heappop(frontier)
    if current == goal:
        return construct_path(...)
    # Expand based on lowest cumulative cost (g)
```

2.2 A* Search

A* is an informed search algorithm that uses both the actual path cost $g(n)$ and an estimated heuristic cost to the goal $h(n)$. It expands nodes based on $f(n) = g(n) + h(n)$. This allows A* to be much more efficient by directing its search towards the goal.

```
# A* Core Logic Snippet
f_cost_neighbor = new_g_cost + manhattan_distance(neighbor, goal)
heapq.heappush(frontier, (f_cost_neighbor, counter, new_g_cost, neighbor))
```

3 Heuristic Function

For the A* algorithm, we utilized the **Manhattan Distance** heuristic. Since the robot is restricted to 4-directional movement (Up, Down, Left, Right), Manhattan Distance perfectly estimates the minimum steps needed to reach the target.

```
def manhattan_distance(pos, goal):
    return abs(pos[0] - goal[0]) + abs(pos[1] - goal[1])
```

Admissibility: A heuristic is admissible if it never overestimates the true cost to reach the goal. Because the minimum possible cost for a single step in our grid is 1 (normal floor), the Manhattan Distance (which assumes a cost of 1 per step) will never exceed the actual path cost (especially since some tiles cost 3). Thus, it is admissible and guarantees A* will find the optimal path.

4 Results Table

The following table demonstrates the performance of both algorithms across the three test maps:

Map	Algorithm	Path Len	Cost	Expanded Nodes	Max Frontier	Time (ms)
MAP_A	UCS	11	15	28	4	0.1064
MAP_A	A*	11	15	21	5	0.1314
MAP_B	UCS	12	12	31	4	0.0894
MAP_B	A*	12	12	15	7	0.0689
MAP_C	UCS	-	-	13	3	0.0337
MAP_C	A*	-	-	13	3	0.0735

5 Path Outputs (Screenshots Equivalent)

Below is the visualization of the optimal path found by the algorithms on the grid (path marked with *****). Both algorithms produced the exact same optimal paths.

5.1 MAP_A

```
S**#....
##*#.##.
..******
.####.#*
...~..#G
```

5.2 MAP_B

```
S~~~~~~~G
*########*
*********
.#.#.#.#.
.........
```

Note: The algorithms successfully avoided the straight path of rough floor (~) to minimize cost.

5.3 MAP_C

Result: No path found (The goal is enclosed by walls).

6 Comparison & Conclusion

Path Cost: Both Uniform Cost Search and A* found the exact same optimal paths with identical costs (15 for MAP_A, and 12 for MAP_B). This validates that our A* implementation is optimal and the Manhattan heuristic is admissible.

Efficiency (Nodes Expanded): A* significantly outperformed UCS in terms of search efficiency. In MAP_A, A* expanded only 21 nodes compared to UCS's 28. The difference was even more substantial in MAP_B, where A* expanded just 15 nodes compared to UCS's 31, saving more than 50% of the search effort.

Conclusion: For this warehouse delivery robot, we strictly recommend the **A* Algorithm**. It guarantees the cheapest path while vastly reducing the number of states the robot's computer needs to evaluate, saving computational resources and battery life.

Project GitHub Repository:
`https://github.com/abdelrahmanhamdy06/Warehouse-Delivery-Robot`